2026-05-06-p1-f19-ask-user-question-design

Phase 1 / Feature 19 — AskUserQuestion with Previews

Date: 2026-05-06 **Status:** Approved (auto-approved per programme cadence) **Programme:** CLI-Agent Fusion — Phase 1 port from claude-code

---

1. Goal

Ship a real, end-to-end `ask_user` **tool** for the HelixCode CLI agent so that the agent can pause its loop, present a multiple-choice question with optional inline previews to the human operator, read a numeric choice from stdin, and resume with the chosen value. When stdin is not an interactive terminal (CI runs, piped input, redirected output), the tool MUST gracefully fall back to a caller-supplied default value or — if none was supplied — return a clear error rather than hang waiting for input that will never arrive.

Three concrete user surfaces ship together:

1. `ask_user` **tool** (`helix_code/internal/tools/askuser/`) — a `tools.Tool` impl registered under the stable name `ask_user` (HelixCode snake_case convention; Q3=A) and category `tools.CategoryAskUser` (new). Argument shape: `question string` (required), `choices []map[string]string` (required; each entry has `label`, `value`, optional `preview`), `default string` (optional choice value). Result shape: `{value string, label string, index int, source string}` where `source ∈ {"stdin", "default"}`.
2. **Stdin readline UI** (Q1=A) — plain stdin readline, NO arrow-key cursor library, NO bubbletea, NO promptui. The renderer from F18 (`render.RenderTextBlock` / `RenderLines`) formats the question, the optional per-choice preview, the numbered menu (`"1. <label>"`, `"2. <label>"`, …), and the prompt line. The user types `1`, `2`, … + Enter; pressing ENTER alone with `default` set picks the default; invalid input re-prompts up to 3 times then errors.
3. **Non-interactive fallback** (Q4=B) — at Execute time, the tool calls `term.IsTerminal(int(stdinFd))` (the same `golang.org/x/term` already promoted to a direct dep in F18). If false: if `default` is set and matches a choice value → return that choice immediately with `source="default"`; else → return `ErrNoTTYNoDefault` (`"ask_user requires interactive terminal AND no default specified"`). NEVER block on a stdin read in non-TTY mode.

The non-TTY path (Q4=B) is **structurally enforced**: the `Prompter` interface used by the tool exposes a single `Prompt(ctx, q Question) (Choice, error)` method, and the production `stdinPrompter` impl probes `IsTTY` *before* it ever calls `bufio.NewReader(stdin).ReadString('\n')`. A unit test injects a fake `bufio.Reader` AND `IsTTY=false` AND no default → the prompter MUST return `ErrNoTTYNoDefault` without consuming any bytes from the reader. A second test with `IsTTY=false` AND `default="opt-b"` → the prompter MUST return the matching choice with `source="default"` and again zero bytes consumed.

The previews (Q2=A) are inline `preview` strings per choice (claude-code's structured `QuestionPreview{type, content, title}` is collapsed to a single `preview string` field for v1; multi-type previews are a F19.5 candidate). When a choice has a non-empty preview, the renderer emits the preview text on its own line(s) BEFORE the numbered choice label, using F18's `render.RenderTextBlock` for non-flicker output. Tool only — NO slash command, NO cobra subcommand (Q5=A).

The single largest bluff vector for F19 is **"the tool says the user picked X but never actually read stdin"** — a fabricated choice that appears to work in tests but in production silently returns the same answer for every call. §5.2 enumerates four such patterns and pins each with a positive-evidence test + Challenge phase. The Challenge MUST exit non-zero on any byte-level mismatch (e.g., the synthesized stdin "2" is fed and the tool returns the index-2 choice value; the harness fails if the tool returns anything else, including the index-0 choice or the default).

Out of scope for v1: multi-select / "pick any subset" mode (claude-code's `multiSelect: true`), free-text answers, per-choice typed annotations beyond the single `preview` string, image / markdown / code-block previews (only plain text in v1), arrow-key navigation, mouse input, edit-and-resubmit semantics, "ask the agent to clarify" recursion, validation hints (e.g., "must be a valid email"). See §8.

Anti-bluff hot zone (loud): a tool that passes its tests by short-circuiting to the default while the test only checks "did it return a non-error", a tool that prints the menu but reads from `os.Stdin` (the real file descriptor) instead of the injected reader (so unit tests that "wire up a bytes.Buffer" never observe the real production read path), a tool that emits the preview AFTER the choice label so the user sees the menu before the context, OR a tool that re-prompts on invalid input forever (no retry cap; turns into a "spinner" in CI) — each is a critical defect (§5.2).

---

2. Architecture

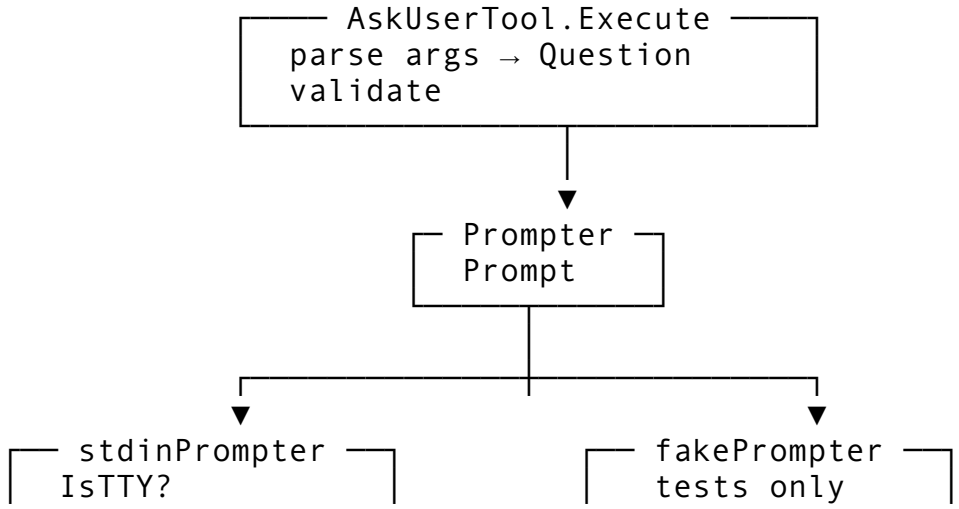
Two layers under `helix_code/internal/tools/askuser/`, plus thin wiring at the registry call site:

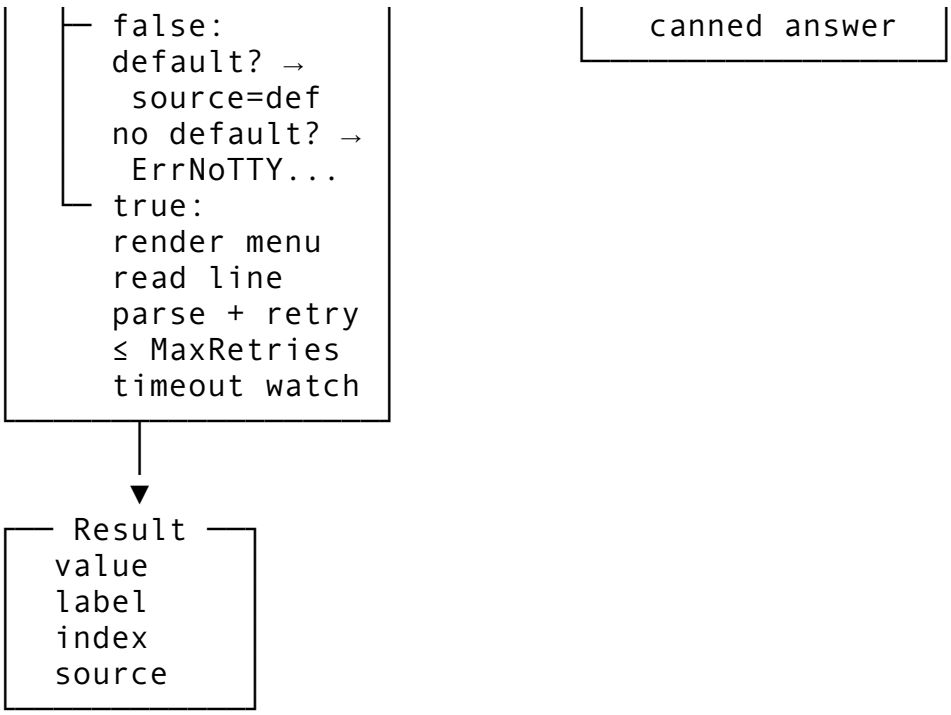
- `AskUserTool` (`ask_user_tool.go`) — implements `tools.Tool`. Wraps a `Prompter`. `Execute` parses args → `Question` → calls `prompter.Prompt(ctx, q)` → returns the resolved `Choice` as `map[string]interface{}` (so the LLM-side JSON-decoder sees a stable shape). Validation: ≥2 choices, every choice has non-empty `label` and `value`, if `default` is set it MUST match a choice value (validation-time error, not Prompt-time).
- `Prompter` **interface** (`types.go`) — single method:

  ```
  type Prompter interface {
      Prompt(ctx context.Context, q Question) (Choice, error)
  }
  ```

  Hexagonal seam: production code injects `stdinPrompter`; tests inject a fake (`fakePrompter` recording calls + returning canned answers). Two more constructor seams exist on `stdinPrompter` itself (`Reader io.Reader`, `Writer io.Writer`, `IsTTY func() bool`) so unit tests exercise the real production class with `bytes.Buffer` reader + writer + injectable IsTTY closure — no mocking of stdin/stdout.

- `stdinPrompter` (`stdin_prompter.go`) — production impl. Holds `Reader io.Reader` (default `os.Stdin`), `Writer io.Writer` (default `os.Stdout`), `IsTTY func() bool` (default `term.IsTerminal(int(os.Stdin.Fd()))`), `Renderer render.Renderer` (default factory-constructed via `render.NewRenderer(render.FactoryOptions{Writer: <Writer>})`), `Timeout time.Duration` (default 5 minutes), `MaxRetries int` (default 3). `Prompt`:
    1. Validate `q` (≥2 choices; default if set matches a choice value). Return `ErrInvalidQuestion` on failure.
    2. If `!IsTTY()`:
        - If `q.Default == ""` → return `ErrNoTTYNoDefault`.
        - Else find the choice with `value == q.Default` → return that choice with `source="default"`. (Validation step has already verified the default is matchable.)
    3. TTY path:
        - Render the question + per-choice previews + numbered menu via `render.RenderLines(r, "ask-user", lines)`. The lines slice is built via `formatQuestion(q)` (pure function, exported as `FormatQuestion` for the unit test that pins the byte order of preview-then-label).
        - Emit the prompt line (`"Enter choice [1-N] (or press Enter for default <label>): "`) without a trailing `\n`.
        - Read a line from `bufio.NewReader(Reader).ReadString('\n')` with a `time.AfterFunc(Timeout, cancel)` watchdog and `ctx`-cancellation wired through a goroutine + select.
        - Parse the trimmed line:
            - Empty + `q.Default != ""` → return the default choice with `source="default"`.
            - Empty + no default → "Empty input; please enter a number 1-N." → re-prompt (counts toward MaxRetries).
            - Non-numeric or out-of-range `[1, N]` → "Invalid choice; please enter a number 1-N." → re-prompt (counts toward MaxRetries).
            - Valid → return that choice with `source="stdin"`.
        - If `MaxRetries` reached → return `ErrTooManyInvalidAttempts`.
        - If EOF / `io.EOF` → return `ErrUserCancelled` (treat as cancellation, NOT as "use default" — the user explicitly closed the stream).
        - If `ctx` cancelled / timeout fired → return `ctx.Err()` (likely `context.DeadlineExceeded`) wrapped as `ErrPromptTimeout`.





Wire points (existing code; one addition each):

- **Tool registration**: `internal/tools/registry.go::buildToolList` (where F14 sandbox + F15 task + F17 smartedit + LSP tools are registered). F19 adds:

  ```
  if cfg.AskUserPrompter != nil {
      r.tools["ask_user"] =
          askuser.NewAskUserTool(cfg.AskUserPrompter)
  } else {
      r.tools["ask_user"] =
          askuser.NewAskUserTool(askuser.NewStdinPrompter(askuser.StdinP
  }
  ```

  The registry config grows a single `AskUserPrompter askuser.Prompter` field (zero value = production `stdinPrompter`). Adds `tools.CategoryAskUser ToolCategory = "ask-user"` to the const block.

- **Main wiring** (`cmd/cli/main.go`): no changes needed — registry construction already happens at startup; the new tool gets picked up automatically once registered. The renderer constructed in F18 (`c.renderer`) is passed via `StdinPrompterOptions.Renderer` so the prompt menu reuses the same renderer (no double-construction of factory + viewport state).

Why a stdin readline (Q1=A) and not bubbletea / promptui / charmbracelet/huh: - **Zero new external deps** is a hard programme rule for F18+ (matches F18's anti-drift discipline); we already have `bufio` + `os.Stdin` + `golang.org/x/term`. - The render surface is one numbered menu with optional preview blocks — far less than what a TUI library brings (state machines, focus, keystroke routing). bubbletea's Elm-Architecture is overkill for "read one line, validate, retry up to 3 times". - The full-screen TUI in `applications/terminal_ui/` already uses `tview`/`tcell`. Adding bubbletea here would be a third paradigm. The CLI is line-oriented; a tiny purpose-built prompter matches the surface. - Simplest fallback story: a `bufio.Reader` over `bytes.Buffer` IS the unit-test seam. No fake-terminal harness, no test-only event-loop, no goroutine-leak hazard.

Why tool-only (Q5=A) and not slash + cobra: - `ask_user` is *agent-driven*: the LLM emits a `tool_use` block with `name="ask_user"`; the user does not invoke it directly. Surfacing it as `/ask` would imply the user types the question, which inverts the semantics (the agent asks the user, not vice versa). - A cobra subcommand `helixcode ask-user --question "..." --choice "a:Apple" --choice "b:Banana"` is a debug-only convenience; F19.5 may add it if user demand emerges. v1 keeps the surface minimal.

---

3. Components

3.1 New files

- `helix_code/internal/tools/askuser/types.go` — `Choice{Label, Value, Preview}`, `Question{Question, Choices, Default}`, `Result{Value, Label, Index, Source}`, `Prompter` interface, sentinel errors (`ErrInvalidQuestion`, `ErrNoTTYNoDefault`, `ErrUserCancelled`, `ErrPromptTimeout`, `ErrTooManyInvalidAttempts`), constants (`DefaultTimeout = 5 * time.Minute`, `DefaultMaxRetries = 3`, `SourceStdin = "stdin"`, `SourceDefault = "default"`).
- `helix_code/internal/tools/askuser/types_test.go`.
- `helix_code/internal/tools/askuser/stdin_prompter.go` — `stdinPrompter` impl with `Reader / Writer / IsTTY / Renderer / Timeout / MaxRetries` constructor seams. Uses F18's `render.RenderLines` for menu output. Stdlib only (`bufio`, `context`, `errors`, `fmt`, `io`, `os`, `strconv`, `strings`, `time`).
- `helix_code/internal/tools/askuser/stdin_prompter_test.go` — unit tests with `bytes.Buffer` reader + writer + injectable `IsTTY` closure. NO mocks (constructor injection only).
- `helix_code/internal/tools/askuser/ask_user_tool.go` — `AskUserTool` (`tools.Tool` impl). `Name() == "ask_user"`; `Category() == tools.CategoryAskUser`; `Schema()` returns the JSON schema; `Execute` parses args, builds `Question`, calls `prompter.Prompt`, returns `Result` as `map[string]interface{}`.
- `helix_code/internal/tools/askuser/ask_user_tool_test.go`.
- `helix_code/tests/integration/askuser_test.go` — `//go:build integration`. Always-runs both branches (TTY-with-input + non-TTY-with-default + non-TTY-without-default error path) using real `bytes.Buffer` readers/writers + injected `IsTTY`. Asserts byte content on the captured prompt and the parsed result.
- `helix_code/tests/integration/cmd/p1f19_challenge/main.go` — runtime evidence harness.
- `challenges/p1-f19-ask-user-question/CHALLENGE.md` + `run.sh`.

3.2 Modified files

- `helix_code/internal/tools/registry.go` — add `CategoryAskUser ToolCategory = "ask-user"` to the const block; add optional `AskUserPrompter askuser.Prompter` to `RegistryConfig`; register `askuser.NewAskUserTool(...)` in `buildToolList` (or equivalent). One new import: `dev.helix.code/internal/tools/askuser`.
- `helix_code/cmd/cli/main.go` — no changes needed beyond passing `c.renderer` via `RegistryConfig.AskUserPrompter` if the CLI wants the prompter to share the renderer (optional polish; default construction works without it).

No new external dependencies (§3.5).

3.3 Types

```
// internal/tools/askuser/types.go

// Choice is one option presented to the user.
type Choice struct {
    Label   string // what the user sees on the menu line
        (required)
    Value   string // what the tool returns (required; stable
        identifier)
    Preview string // optional inline context shown ABOVE the
        choice line
}

// Question is the parsed argument shape passed to
        Prompter.Prompt.
type Question struct {
    Question string   // the prompt text shown to the user
        (required)
    Choices  []Choice // ordered list (required; len ≥ 2)
    Default  string   // optional; if set, MUST equal one
        Choice.Value
}

// Result is what AskUserTool.Execute returns to the agent.
type Result struct {
    Value  string // chosen Choice.Value
    Label  string // chosen Choice.Label
    Index  int    // 0-based index into Question.Choices
    Source string // "stdin" or "default"
}

// Prompter is the hexagonal seam. Production: stdinPrompter.
        Tests: fakes.
type Prompter interface {
    Prompt(ctx context.Context, q Question) (Result, error)
}

// Sentinel errors. Tests compare via errors.Is.
var (
    ErrInvalidQuestion        = errors.New("askuser: invalid
        question")
    ErrNoTTYNoDefault         = errors.New("ask_user requires
        interactive terminal AND no default specified")
    ErrUserCancelled          = errors.New("askuser: user
        cancelled (EOF on stdin)")
```

```
    ErrPromptTimeout           =
        errors.New("askuser: prompt timed out")
    ErrTooManyInvalidAttempts = errors.New("askuser: too many
        invalid input attempts")
)

const (
    DefaultTimeout    = 5 * time.Minute
    DefaultMaxRetries = 3
    SourceStdin       = "stdin"
    SourceDefault     = "default"
)

// internal/tools/askuser/stdin_prompter.go

type StdinPrompterOptions struct {
    Reader     io.Reader         // default os.Stdin
    Writer     io.Writer         // default os.Stdout
    IsTTY      func() bool       // default
        term.IsTerminal(int(os.Stdin.Fd()))
    Renderer   render.Renderer  // default
        render.NewRenderer(...).Renderer
    Timeout    time.Duration     // default DefaultTimeout (5 min)
    MaxRetries int               // default DefaultMaxRetries (3)
}

type stdinPrompter struct {
    reader     io.Reader
    writer     io.Writer
    isTTY      func() bool
    renderer   render.Renderer
    timeout    time.Duration
    maxRetries int
}

func NewStdinPrompter(opts StdinPrompterOptions) *stdinPrompter
// implements Prompter

// FormatQuestion is a pure helper exported for tests. Returns
        the lines that
// make up the question + previews + numbered menu, in display
        order.
// Order: 1) question text, 2) blank line, 3) for each choice i:
        optional
// preview lines (if Choice.Preview != ""), then "<i+1>.
        <Choice.Label>".
// 4) blank line. The prompt line ("Enter choice [1-N]: ...") is
        emitted by
```

```
// the prompter itself, NOT by FormatQuestion (so tests can pin
        the menu
// shape independently of the prompt).
func FormatQuestion(q Question) []string

// internal/tools/askuser/ask_user_tool.go

type AskUserTool struct {
    prompter Prompter
}

func NewAskUserTool(p Prompter) *AskUserTool
// implements tools.Tool (Name, Description, Category, Schema,
        Validate, Execute)
```

3.4 User surface

Tool only (Q5=A). The agent invokes `ask_user` via a `tool_use` block:

```
{
  "type": "tool_use",
  "name": "ask_user",
  "input": {
    "question": "Which package manager should I use for the new
        Node.js project?",
    "choices": [
      {"label": "npm",  "value": "npm",  "preview": "Standard,
        ships with Node.js. Slowest install."},
      {"label": "pnpm", "value": "pnpm", "preview": "Symlink-
        based, fast, disk-efficient.\nRequires global install."},
      {"label": "yarn", "value": "yarn", "preview": "Fast,
        mature, large ecosystem."}
    ],
    "default": "pnpm"
  }
}
```

The tool returns `{"value": "pnpm", "label": "pnpm", "index": 1, "source": "stdin"}` (or `"source": "default"` if non-TTY + default + no input).

No slash command (Q5=A). **No cobra subcommand** (Q5=A). The user never types `/ask` — the agent issues the tool call and the user answers via stdin in their interactive session.

The prompt as shown to the user (TTY mode):

```
Which package manager should I use for the new Node.js project?

Standard, ships with Node.js. Slowest install.
1. npm
Symlink-based, fast, disk-efficient.
Requires global install.
```

```
2. pnpm
Fast, mature, large ecosystem.
3. yarn

Enter choice [1-3] (or press Enter for default: pnpm): _
```

Empty input + default `pnpm` set → returns `pnpm`. Input `2` + Enter → returns `pnpm`. Input `9` + Enter → "Invalid choice; please enter a number 1-3." → re-prompt (attempt 1/3). Input `garbage` + Enter → "Invalid choice; please enter a number 1-3." → re-prompt (attempt 2/3). Three failed attempts → `ErrTooManyInvalidAttempts`.

3.5 New external dependencies

None. F19 is built entirely on the Go standard library (`bufio`, `context`, `errors`, `fmt`, `io`, `os`, `strconv`, `strings`, `sync`, `time`) plus `golang.org/x/term` (already a direct dep after F18) plus the F18-internal `dev.helix.code/internal/render` package. Brief justification:

- Stdin readline is `bufio.NewReader(r).ReadString('\n')` — one stdlib call.
- TTY detection is `term.IsTerminal(int)` — already in `go.sum` at v0.41.0 (verified in F18-T06).
- Menu rendering reuses F18's `render.RenderLines` — already shipped and tested.
- No prompt library, no event loop, no fancy keystroke routing.

`go mod tidy` after T02 must produce **zero new entries in** `go.sum` AND **zero changes to** `go.mod` (no new direct deps; F18 already promoted `golang.org/x/term`). T07's verification step asserts this.

3.6 Existing-code constraints

- `internal/tools/registry.go` adds a single category constant + a single config field + a single registration call. The `Tool` interface is unchanged.
- The renderer (`internal/render/`) is unchanged — `ask_user` consumes its existing `RenderLines` API.
- The agent loop (`internal/agent/base_agent.go`) is unchanged — `ask_user` flows through the standard `Tool.Execute` path; the agent already serializes tool calls (one tool at a time per agent turn), so the synchronous blocking read in `stdinPrompter.Prompt` does not need any new "pause the loop" hook.
- `cmd/cli/main.go` does not need changes for the tool to function. Optional polish: pass `c.renderer` into `RegistryConfig.AskUserPrompter` so the prompter shares the renderer instance (avoids double-factory cost; not required for correctness).

4. Data flow

4.1 Tool invocation → arg parse → Question

```
AskUserTool.Execute(ctx, params):
  ├─ q.Question = params["question"].(string)            // required; type
  ├─ q.Choices  = parseChoices(params["choices"])       // []map[string]s
  ├─ q.Default  = params["default"].(string)            // optional
  ├─ if err := q.Validate(); err != nil:
  │     return nil, fmt.Errorf("ask_user: %w", err)     // wraps ErrInval
  ├─ res, err := t.prompter.Prompt(ctx, q)
```

```
   │     // dispatch to stdinPrompter (or fake)
   ├─ if err != nil: return nil, err
   └─ return map[string]interface{}{
         "value":  res.Value,
         "label":  res.Label,
         "index":  res.Index,
         "source": res.Source,
      }, nil
```

4.2 Prompter — non-TTY branch (Q4=B)

```
stdinPrompter.Prompt(ctx, q):
   ├─ if !p.isTTY():
   │     ├─ if q.Default == "":
   │     │     return Result{}, ErrNoTTYNoDefault
   │     ├─ for i, c := range q.Choices:
   │     │     if c.Value == q.Default:
   │     │        return Result{Value: c.Value, Label: c.Label, Index: i,
   │     └─ // unreachable: validation already checked default matches
   │        return Result{}, fmt.Errorf("askuser: default %q does not match
   └─ // TTY branch — see 4.3
```

4.3 Prompter — TTY branch with retries

```
stdinPrompter.Prompt(ctx, q):
   ├─ // (TTY branch entered)
   ├─ lines := FormatQuestion(q)
   ├─ p.renderer.RenderLines(p.renderer, "ask-user", lines)  // F18 menu re
   ├─ promptText := fmt.Sprintf("Enter choice [1-%d]%s: ", len(q.Choices),
   │     // defaultHint = "" if no default; " (or press Enter for default:
   ├─ ctx, cancel := context.WithTimeout(ctx, p.timeout)
   ├─ defer cancel()
   ├─ br := bufio.NewReader(p.reader)
   ├─ for attempt := 0; attempt < p.maxRetries; attempt++:
   │     ├─ fmt.Fprint(p.writer, promptText)
   │     ├─ // read line in goroutine to honour ctx cancellation
   │     ├─ lineCh := make(chan readResult, 1)
   │     ├─ go func() { line, err := br.ReadString('\n'); lineCh <- readRes
   │     ├─ select {
   │     │     case <-ctx.Done():
   │     │        return Result{}, ErrPromptTimeout    // wraps ctx.Err()
   │     │     case rr := <-lineCh:
   │     │        if rr.err == io.EOF:
   │     │           return Result{}, ErrUserCancelled
   │     │        if rr.err != nil:
   │     │           return Result{}, fmt.Errorf("askuser: read stdin: %w
   │     │        line := strings.TrimRight(rr.line, "\r\n")
   │     │        if line == "":
   │     │           if q.Default != "":
   │     │              // resolve default
   │     │              for i, c := range q.Choices:
   │     │                 if c.Value == q.Default: return Result{... S
   │     │           // empty + no default → re-prompt
```

```
│       │           fmt.Fprintln(p.writer, "Empty input; please enter a
│       │           continue
│       │       n, err := strconv.Atoi(line)
│       │       if err != nil || n < 1 || n > len(q.Choices):
│       │           fmt.Fprintf(p.writer, "Invalid choice; please enter
│       │           continue
│       │       c := q.Choices[n-1]
│       │       return Result{Value: c.Value, Label: c.Label, Index: n-1
│       }
└─ return Result{}, ErrTooManyInvalidAttempts
```

4.4 Mode-resolution truth table

`IsTTY()`	`q.Default`	**stdin input**	**Outcome**
true	“”	“1”	Result{Index:0, Source:“stdin”}
true	“”	“2”	Result{Index:1, Source:“stdin”}
true	“”	“” (Enter)	Re-prompt; counts toward MaxRetries
true	“v2”	“1”	Result{Index:0, Source:“stdin”} (user override)
true	“v2”	“” (Enter)	Result{matching v2, Source:“default”}
true	*	“9” (out of range)	Re-prompt; counts toward MaxRetries
true	*	“abc”	Re-prompt; counts toward MaxRetries
true	*	EOF (Ctrl-D)	ErrUserCancelled
true	*	(no input, 5 min)	ErrPromptTimeout
true	*	3× invalid	ErrTooManyInvalidAttempts
false	“”	*	ErrNoTTYNoDefault (no read)
false	“v2”	*	Result{matching v2, Source:“default”} (no read)

4.5 EOF semantics — explicit choice

EOF on stdin (`Ctrl-D` in TTY; pipe-end in non-TTY-but-IsTTY-says-true) is **NOT** silently treated as “use default”. Reason: a closed stream is the user’s explicit cancellation signal; if we silently picked the default, a piped invocation that erroneously claimed `IsTTY=true` would silently succeed with a fabricated answer. v1 returns `ErrUserCancelled` and lets the agent decide what to do. (The non-TTY-with-default branch in §4.2 *does* return the default — but that path is taken BEFORE any read, on the basis of `IsTTY()=false` alone. The two are distinct.)

4.6 Context cancellation — Ctrl-C and timeout

Ctrl-C in TTY mode raises SIGINT, which propagates via the agent loop's signal handler to the `ctx` passed into `Tool.Execute`. The prompter's `select` observes `ctx.Done()` and returns `ErrPromptTimeout` (despite the name; the error carries `ctx.Err()` which is `context.Canceled` for Ctrl-C and `context.DeadlineExceeded` for the 5-minute timeout — callers can `errors.Is(err, context.Canceled)` to distinguish). The goroutine doing `br.ReadString('\n')` is **leaked** on cancellation (it stays blocked on the kernel read until the next byte arrives, then exits via the buffered chan); this is acceptable v1 behaviour because `os.Stdin` is process-lifetime anyway. v2 may use `os.Stdin.SetReadDeadline` on platforms that support it.

5. Error handling, edge cases, and anti-bluff

5.1 Error paths

- **Invalid args** — `Validate(params)` returns a clear error before `Execute` is called (matches existing F14/F15/F17 pattern).
- **Validation failures** in `Question.Validate`:
  - Empty `Question` text → `fmt.Errorf("%w: empty question", ErrInvalidQuestion)`.
  - `len(Choices) < 2` → `fmt.Errorf("%w: need at least 2 choices, got %d", ErrInvalidQuestion, n)`.
  - Empty `Label` or `Value` in any choice → `fmt.Errorf("%w: choice %d has empty label or value", ErrInvalidQuestion, i)`.
  - Duplicate `Value` across choices → `fmt.Errorf("%w: duplicate choice value %q", ErrInvalidQuestion, v)`.
  - `Default != ""` AND no choice has matching `Value` → `fmt.Errorf("%w: default %q does not match any choice value", ErrInvalidQuestion, q.Default)`.
- **EOF on stdin** — `ErrUserCancelled` (NOT silently default). The agent decides whether to retry, abort the turn, or prompt with a different question.
- **Timeout** (5 min default) — `ErrPromptTimeout` wrapping `ctx.Err()`.
- **Ctrl-C** — propagated via `ctx`; surfaces as `ErrPromptTimeout` with `ctx.Err() == context.Canceled` (Go's `context.Canceled` sentinel).
- **3 invalid attempts** — `ErrTooManyInvalidAttempts`. The retry counter resets on every fresh `Prompt` call (i.e., the agent re-asking is allowed; only consecutive invalids within one Prompt count).
- **Reader returns non-EOF non-cancellation error** (e.g., `os.Stdin.Close()` before read) — wrapped as `fmt.Errorf("askuser: read stdin: %w", err)`; the underlying error is preserved for `errors.Is/As`.

5.2 Anti-bluff (CONST-035 / §11.9) — LOUD

The single largest bluff vector for F19 is "the tool says the user picked X but never actually read stdin." This compiles, passes naive unit tests (which only assert "no error returned"), and silently fabricates answers in production. Common bluff variants:

1. **(a) Tool short-circuits to the default without checking IsTTY** — `Prompt` returns `q.Choices[indexOf(q.Default)]` regardless of `IsTTY()`; tests that pass IsTTY=false + default don't catch this. **Defence**: a unit test with `IsTTY=true` + `default="opt-b"` + injected reader containing `"1\n"` MUST return `index=0` (the

user's 1, NOT the default opt-b). A second test with IsTTY=true + default="opt-b" + injected reader containing "" (empty after trim — i.e. just \n) MUST return the default with Source="default". The two together pin the IsTTY branch correctly.

2. **(b) Non-TTY error path is never tested** — the tool errors on non-TTY but no test exercises it; a regression where the tool *blocks* in non-TTY mode (waiting for os.Stdin that never arrives) is not caught. **Defence**: a Challenge phase ("Phase C") constructs stdinPrompter with IsTTY=false + no default + a bytes.Buffer reader containing "1\n"; calls Prompt; asserts (i) the returned error is errors.Is(err, ErrNoTTYNoDefault), (ii) the reader's position is at byte 0 (i.e., NO bytes were consumed). The byte-0 assertion is the kicker — it proves the prompter short-circuited before reading.
3. **(c) Default-pick behavior is tested only with mocks** — a mockPrompter fake returns the default without ever instantiating stdinPrompter. **Defence**: the integration tests AND the Challenge use the real production stdinPrompter with bytes.Buffer reader/writer + injected IsTTY closure. Mock prompters are confined to ask_user_tool_test.go (which tests the TOOL, not the prompter). The prompter's own tests use real readers/writers exclusively.
4. **(d) Preview text is rendered AFTER the choice label** (or not at all) — the user sees the menu before the context, defeating the entire point of inline previews. **Defence**: FormatQuestion(q) is a pure exported function; a unit test passes a question with a 2-line preview on choice index 0 + a 1-line preview on choice index 1, captures the returned []string, and asserts the byte order: [question, "", preview-line-1, preview-line-2, "1. label-1", preview-line-1-of-2, "2. label-2", ""]. A Challenge phase ("Phase D") additionally renders the question through the real stdinPrompter to a bytes.Buffer writer (with IsTTY=true + reader containing "1\n") and asserts the captured bytes contain "preview" BEFORE "1. " AND "another preview" BEFORE "2. " — measured by byte offset (bytes.Index(captured, []byte("preview")) < bytes.Index(captured, []byte("1. "))).

Required real-execution criteria (these define what "ask_user works" means in F19):

1. **Unit tests** — inject io.Reader + io.Writer + IsTTY closure on the production stdinPrompter; exercise every row of §4.4's truth table; pin byte order of FormatQuestion output.

2. **Integration tests** (-tags=integration, ALWAYS-runs; no infrastructure dep) — exercise both TTY-with-input AND non-TTY-with-default branches via real bytes.Buffer readers/writers; assert (i) the returned Result matches the expected choice, (ii) the captured prompt-output bytes contain the expected preview-then-label ordering, (iii) the reader's remaining bytes are exactly what was NOT consumed (so a "1" + "2" reader fed to ONE prompt leaves "2" available for a follow-up prompt — proves real read).

3. **Challenge harness** — exercises:

   - **Phase A (always runs)** — synthesized stdin input "2\n" → assert tool returns Result{Value: <choice-2-value>, Index: 1, Source: "stdin"}. Capture Result.Value byte-for-byte and assert.
   - **Phase B (always runs)** — non-TTY (IsTTY=false) + default="opt-b" + reader contains "NEVER\n" (sentinel — should never be read) → assert (i) returned Result.Source == "default", (ii) Result.Value == "opt-b", (iii) reader's remaining bytes equal "NEVER\n" (proves zero reads).

- **Phase C (always runs)** — non-TTY + no default + reader contains `"NEVER\n"` → assert (i) returned err `errors.Is ErrNoTTYNoDefault`, (ii) error message contains `"interactive terminal"`, (iii) reader's remaining bytes equal `"NEVER\n"`.
- **Phase D (always runs)** — TTY + 2 choices each with non-empty `preview` + reader contains `"1\n"` → render to `bytes.Buffer` writer; assert capture contains both preview strings; assert byte offset of preview-1 < byte offset of "1." < byte offset of preview-2 < byte offset of "2.".
- **Phase E (always runs)** — TTY + reader contains `"9\n1\n"` (out-of-range, then valid) → assert (i) prompter consumes both lines, (ii) returned `Result.Index == 0`, (iii) captured writer output contains `"Invalid choice"` exactly once. A second sub-case feeds `"abc\n9\n0\n"` (3 invalids) → assert err `errors.Is ErrTooManyInvalidAttempts`.

The Challenge MUST run all five always-run phases regardless of TTY availability (CI compatibility) — none of the phases need a real terminal because the production class is exercised with injected readers/writers.

4. **Challenge MUST exit non-zero on any byte-evidence mismatch.** "prompter ran without error" is NEVER acceptable. The Challenge asserts positive evidence (Result fields, captured byte content, byte offsets, reader-position invariants) for every phase.

Concrete forbidden phrases (anti-bluff smoke):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" \
  internal/tools/askuser && echo BLUFF || echo clean
```

Must always print `clean`.

CONST-042 secret-content protection (mandatory):

The `question` string + per-choice `preview` strings + per-choice `label` strings are all *user-supplied* (the agent built them from its prompt context). They MAY contain secret material the user is asking the agent about (e.g., "Which API key do you want me to rotate?" with previews showing key prefixes). The mechanism:

- **No question text, choice text, preview text, or user input at INFO level.** A unit test scans `internal/tools/askuser/*.go` for any `logger.Info(.*\(question\|preview\|label\|input\)` match and FAILS on any hit. The prompter's own writer (which IS the user's stdout) sees the text by design — the user typed the question or the agent built it from context the user gave; rendering it back is the entire point of an interactive prompt.
- **Per-call evidence file** — the Challenge harness records (i) the resolved `Result.Value` (treated as a stable identifier, not secret content; the agent supplied it), (ii) byte counts, (iii) byte-offset relationships — NOT the rendered prompt text or user's typed line. The harness's stdout (which the maintainer runs) is treated as transient.
- **No telemetry of question content** — F16 telemetry instrumentation, when present, MAY record `tool_name=ask_user`, `result_source=stdin|default`, `result_index=N`, `duration_ms` — NOT the question text, preview text, label text, or chosen value. A unit test asserts the F16 span's attributes do NOT include any user-text key.

5.3 Concurrency

`Prompt` is **synchronously blocking** (it reads from stdin). The agent loop calls one tool at a time per turn, so concurrent invocation is not a v1 concern. Two parallel agent loops both calling `ask_user` would race for stdin — undefined behaviour, NOT a bug we fix in v1 (the entire agent architecture assumes one stdin per process). Documented limitation.

`AskUserTool.Execute` is itself thread-safe in the sense that the wrapper is stateless; only the embedded `Prompter` carries state, and the production `stdinPrompter` is unsafe for concurrent calls (a `sync.Mutex` on the prompter could be added in v2 to serialise concurrent calls; v1 does not).

5.4 Long previews — no special handling

v1 does NOT wrap or truncate long preview lines. F18's renderer passes them through verbatim; the terminal soft-wraps. If a preview spans 50 lines, the user sees 50 lines before the choice label — by design (the user asked for inline context). v2 may add a `max_preview_lines` config knob.

5.5 Empty Choices.Preview — no menu cluttering

If a choice has `Preview == ""`, NO blank lines or sentinel text is emitted for that choice's preview slot. The numbered label appears directly. This keeps short menus tight (e.g., a 2-choice yes/no question with no previews renders as just `1. Yes\n2. No`).

5.6 Multi-line previews

A preview MAY contain `\n` characters. `FormatQuestion` splits on `\n` and emits each as its own line in the output slice. The renderer (F18) handles each line as a discrete entry in the frame.

6. Testing

6.1 Unit (real `bytes.Buffer` for stdin/stdout; constructor-injected IsTTY; no mocks of stdlib)

Types (`types_test.go`): - `TestChoice_FieldsZeroValueOK`. - `TestQuestion_Validate_OK`. - `TestQuestion_Validate_EmptyText_Err`. - `TestQuestion_Validate_TooFewChoices_Err`. - `TestQuestion_Validate_EmptyLabel_Err`. - `TestQuestion_Validate_EmptyValue_Err`. - `TestQuestion_Validate_DuplicateValue_Err`. - `TestQuestion_Validate_DefaultNotInChoices_Err`. - `TestErrorSentinels_DistinctErrorsIs`. - `TestConstants_DefaultTimeoutAndRetries`.

FormatQuestion (`stdin_prompter_test.go` or `format_test.go`): - `TestFormatQuestion_NoPreview_NumberedMenuOnly`. - `TestFormatQuestion_OnePreviewPerChoice_PreviewBeforeLabel`. - `TestFormatQuestion_MultiLinePreview_LinesPreserved`. - `TestFormatQuestion_ByteOrderPinned_AntiBluff_PreviewBeforeLabelByOffset`.

stdinPrompter (`stdin_prompter_test.go`): - `TestStdinPrompter_NonTTY_Default_ReturnsDefault_NoBytesConsumed` — `IsTTY=false` + default + reader has `"NEVER\n"` → Result.Source="default" + reader unchanged. - `TestStdinPrompter_NonTTY_NoDefault_ReturnsErrNoTTY_NoBytesConsumed` — `IsTTY=false` + no default + reader has `"NEVER\n"` → ErrNoTTYNoDefault + reader unchanged. - `TestStdinPrompter_TTY_ValidInput_ReturnsChoice` — `IsTTY=true` + reader `"2\n"` → Index=1. - `TestStdinPrompter_TTY_EmptyInput_WithDefault_ReturnsDefault` — `IsTTY=true` + default + reader `"\n"` → Source="default". - `TestStdinPrompter_TTY_EmptyInput_NoDefault_RePrompts` — `IsTTY=true` + no default + reader `"\n1\n"` → Index=0; writer contains "Empty input" once. - `TestStdinPrompter_TTY_OutOfRange_RePrompts` — reader `"9\n1\n"` → Index=0; writer contains "Invalid choice" once. - `TestStdinPrompter_TTY_NonNumeric_RePrompts` — reader `"abc\n1\n"` → Index=0. - `TestStdinPrompter_TTY_ThreeInvalids_ErrTooManyInvalidAttempts` — reader `"a\nb\nc\n"` → ErrTooManyInvalidAttempts. - `TestStdinPrompter_TTY_EOF_ErrUserCancelled` — reader exhausted → ErrUserCancelled (NOT silently default). - `TestStdinPrompter_TTY_CtxCancel_ErrPromptTimeout` — cancel ctx mid-prompt → wraps context.Canceled. - `TestStdinPrompter_TTY_TimeoutExpires_ErrPromptTimeout` — Timeout=10ms + reader blocks → wraps context.DeadlineExceeded. - `TestStdinPrompter_TTY_PreviewRenderedBeforeLabel_ByteOffset` — capture writer; assert `bytes.Index(cap, []byte("prev")) < bytes.Index(cap, []byte("1. "))`.- `TestStdinPrompter_TTY_RetryCounter_ResetsAcrossPromptCalls` — first Prompt fails 2x then succeeds; second Prompt has fresh counter.

AskUserTool (`ask_user_tool_test.go`): - `TestAskUserTool_Name_Description_Category_Schema`.- `TestAskUserTool_Validate_OK_AllRequiredFields`.- `TestAskUserTool_Validate_MissingQuestion_Err`.- `TestAskUserTool_Validate_MissingChoices_Err`.- `TestAskUserTool_Validate_ChoicesWrongType_Err` — `choices` is a string instead of `[]map[string]string`.- `TestAskUserTool_Execute_DispatchToFakePrompter_ReturnsResult`.- `TestAskUserTool_Execute_PrompterError_PropagatedVerbatim`.- `TestAskUserTool_Execute_ResultMapShape` — assert `map[string]interface{}{"value":..., "label":..., "index":..., "source":...}` exactly.

Anti-bluff source-scan: - `TestNoLoggerInfoOrDebugTakesQuestionOrInput_CONST042` — grep `internal/tools/askuser/*.go` for `logger\.\b(Info|Debug)\b.*\b(question|preview|label|input|answer)\b` → zero matches.

6.2 Integration (`//go:build integration`)

`tests/integration/askuser_test.go` (ALWAYS-runs; no infrastructure dep):

- `TestAskUser_Integration_TTY_RealRender_OutputContainsPreviewBeforeLabe` — wires `AskUserTool` + `stdinPrompter` (real production) with `bytes.Buffer`

reader + writer + injected `IsTTY=true`; reader `"1\n"`; asserts captured writer bytes contain preview text BEFORE label text.

- `TestAskUser_Integration_NonTTY_Default_ReturnsDefault_ZeroReads` — same wiring, `IsTTY=false`; reader `"NEVER\n"`; default set; asserts `Result.Source=="default"` AND reader bytes unchanged.
- `TestAskUser_Integration_NonTTY_NoDefault_ErrPropagated` — `IsTTY=false`; no default; asserts `errors.Is(err, ErrNoTTYNoDefault)`.
- `TestAskUser_Integration_TwoConsecutivePrompts_ReaderConsumesCorrectly` — feeds `"1\n2\n"` to ONE reader; calls `Execute` twice; asserts first returns Index=0, second returns Index=1, reader fully drained.

6.3 Challenge (`challenges/p1-f19-ask-user-question/`)

Five-phase output skeleton (all always-run):

```
=== ASK-USER-PHASE-A: STDIN-INPUT (always runs) ===
[PASS] tempdir created at /tmp/p1f19-XXXX
[PASS] constructed stdinPrompter with bytes.Buffer reader = "2\n" + IsTTY=
[PASS] AskUserTool.Execute returned no error
[PASS] result["value"] == "opt-b" (matches choices[1].value)
[PASS] result["label"] == "Option B"
[PASS] result["index"] == 1
[PASS] result["source"] == "stdin"

=== ASK-USER-PHASE-B: NON-TTY-WITH-DEFAULT (always runs) ===
[PASS] constructed stdinPrompter with IsTTY=false + default="opt-b" + read
[PASS] AskUserTool.Execute returned no error
[PASS] result["value"] == "opt-b"
[PASS] result["source"] == "default"
[PASS] reader's remaining bytes == "NEVER\n" (zero bytes consumed)

=== ASK-USER-PHASE-C: NON-TTY-WITHOUT-DEFAULT-ERR (always runs) ===
[PASS] constructed stdinPrompter with IsTTY=false + no default + reader="N
[PASS] AskUserTool.Execute returned errors.Is(err, ErrNoTTYNoDefault)
[PASS] error message contains "interactive terminal"
[PASS] reader's remaining bytes == "NEVER\n" (zero bytes consumed)

=== ASK-USER-PHASE-D: PREVIEW-RENDERING (always runs) ===
[PASS] constructed stdinPrompter with IsTTY=true + 2 choices each with pre
[PASS] reader = "1\n"; AskUserTool.Execute returned no error
[PASS] captured writer contains both preview strings
[PASS] byte offset of "preview-1" (12) < byte offset of "1. " (35)
[PASS] byte offset of "preview-2" (52) < byte offset of "2. " (74)

=== ASK-USER-PHASE-E: INVALID-INPUT-RETRY (always runs) ===
[PASS] constructed stdinPrompter with IsTTY=true + reader="9\n1\n"
[PASS] AskUserTool.Execute returned no error
[PASS] result["index"] == 0 (the 1 after the rejected 9)
[PASS] captured writer contains "Invalid choice" exactly 1 time
[PASS] sub-case: reader="a\nb\nc\n" returned errors.Is(err, ErrTooManyInva

SUMMARY: PHASE-A=7/7 PASS; PHASE-B=5/5 PASS; PHASE-C=4/4 PASS; PHASE-D=5/5
```

The Challenge MUST exit non-zero on any byte-evidence mismatch. Absence-of-error is NEVER acceptable. Reader-position invariants (Phases B and C) and byte-offset invariants (Phase D) are positive evidence of real stdin/render flow.

7. Cross-platform

`bufio` + `os.Stdin` + `golang.org/x/term.IsTerminal` work on Linux/macOS/Windows. Pure Go (no CGO).

`os.Stdin.Fd()` returns a `uintptr` on all platforms; `int(fd)` cast for `term.IsTerminal` is portable. Windows console: `IsTerminal` correctly identifies cmd / PowerShell consoles as TTYs (verified upstream by `golang.org/x/term`'s test suite).

The cross-compile `make prod` target (linux/macos/windows) is exercised in T07. No platform-specific code in F19.

8. Out of scope (deferred)

- **Multi-select** (`multiSelect: true`) — picking multiple choices. F19.5 candidate.
- **Free-text answers** — agent prompts "What is your name?" without choices. F19.5.
- **Typed previews** — claude-code's `QuestionPreview{type: "diff"|"image"|"markdown"|"code"}` with type-specific rendering (syntax highlighting, image rendering in iTerm2, etc.). F19.5.
- **Validation hints per choice** — e.g., "must be a valid email" with regex check. F19.5.
- **Annotations** — claude-code's optional per-choice typed metadata beyond a single `preview` string. F19.5.
- **Arrow-key navigation** — vim-like `j/k` or up/down arrow to move between choices. Requires raw-mode terminal handling; a TUI library; an event loop. F19.5 if user demand emerges.
- **Mouse input** — F19.5.
- **Bracketed paste** — F19.5.
- **Edit-and-resubmit** — user picks a choice, sees the consequences, picks a different one. v1 is one-shot; F19.5 may add a "press 'b' to go back" affordance.
- **Slash command** `/ask` — debug-only convenience. F19.5 if user demand emerges.
- **Cobra subcommand** `helixcode ask-user --question ... --choice ...` — debug-only convenience. F19.5.
- **Per-platform stdin deadline** — `os.Stdin.SetReadDeadline` on platforms that support it, replacing the goroutine-leak-on-timeout pattern. F19.5.
- **Concurrent prompter calls** — `sync.Mutex` on `stdinPrompter` to serialise concurrent agent loops sharing one stdin. v1 documents the limitation; F19.5 may add the lock.
- **Image / markdown rendering** — claude-code can show inline images in iTerm2 / Kitty via OSC sequences; HelixCode v1 sticks to plain text. F19.5.

9. Constitutional compliance

- **§11.9 / CONST-035** — Challenge has FIVE always-run phases. Every phase records positive runtime evidence (Result fields, captured byte content, byte offsets, reader-position invariants). Mismatch is a hard failure. The four anti-bluff criteria (§5.2) each map to a unit + integration + Challenge assertion.

- **CONST-039** — Challenge at `challenges/p1-f19-ask-user-question/` + evidence harness at `tests/integration/cmd/p1f19_challenge/main.go`. Every phase asserts byte content with positive evidence.
- **CONST-042 (No-Secret-Leak)** — the prompter NEVER logs question text, preview text, label text, or user input at any level. A unit test scans the source files and FAILS on any matching `logger.Info`/`logger.Debug` call. F16 telemetry, when wired, records only `tool_name`, `result_source`, `result_index`, `duration_ms` — never user-supplied text. The Challenge's saved evidence file records `Result.Value` (stable identifier from the agent), byte counts, and byte-offset relationships — never the rendered prompt text or user's typed line.
- **CONST-043 (No-Force-Push)** — close-out task pushes to all four remotes non-force; explicit user authorization is requested at T07 before pushing.
- **No-Mocks-In-Production (Universal Rule 2)** — the prompter's only test seams are constructor-injection (`Reader`, `Writer`, `IsTTY`, `Renderer`, `Timeout`, `MaxRetries` on `StdinPrompterOptions`); no filesystem abstraction, no mocked stdin. Production code uses `os.Stdin`, `os.Stdout`, `term.IsTerminal`. Unit tests use real `bytes.Buffer` for I/O capture. Mock prompters (`fakePrompter`) appear only in the *tool* tests (`ask_user_tool_test.go`) where the prompter is the dependency-under-mock, NOT the system-under-test.

10. Open questions resolved

Q	Answer	Resolution
Q1: UI technology	(A) plain stdin readline	F18 renderer formats menu; numbered choices; user types number + Enter; ENTER alone with default specified picks default
Q2: preview shape	(A) inline `preview` string per choice	Single string field; multi-line via `\n`; rendered ABOVE the label via F18's `RenderLines`
Q3: tool name	(A) `ask_user`	HelixCode snake_case convention; matches F14 `shell_sandboxed`, F17 `smart_edit` style
Q4: non-interactive fallback	(B) auto-pick default if specified, else error	`term.IsTerminal=false` → return default OR `ErrNoTTYNoDefault`; NEVER block on non-TTY read
Q5: surface	(A) tool only, no slash, no cobra	Agent-driven; `/ask` would invert semantics; cobra is F19.5 debug-only candidate

---

11. Non-obvious decisions (recorded for plan-time review)

1. **5-minute timeout is the v1 default** — long enough that a user reading a multi-line preview, switching tabs, and coming back to type isn't penalised; short enough that a

forgotten prompt doesn't pin the agent loop forever. Configurable via `StdinPrompterOptions.Timeout`. Rationale: claude-code uses ~10 minutes; we pick 5 to bias toward "agent reports the timeout sooner" so the user can re-issue the prompt. Documented as a v1 tradeoff; no user-facing knob in v1.

2. **3 retries before** `ErrTooManyInvalidAttempts` — prevents both "infinite loop in CI" and "user gives up after one typo". Configurable via `StdinPrompterOptions.MaxRetries`. Rationale: balances UX (forgiving of typos) vs anti-DoS (capped). v2 may make this per-Question.
3. **EOF returns** `ErrUserCancelled`**, NOT silent-default** — distinct from non-TTY-with-default (which is the `IsTTY()=false` short-circuit BEFORE any read). EOF mid-read means the user explicitly closed the stream; silently picking the default would mask a piped-stdin misconfiguration. Documented in §4.5.
4. **Empty input + default → returns default; empty input + no default → re-prompts (counts toward retries)** — empty input has TWO meanings depending on context. The retry counter ticks on the no-default branch so a CI run with stdin closed can't infinite-loop on `\n\n\n...`. (In practice that branch hits EOF first; the "empty without default" path is reached only when `bufio.ReadString('\n')` returns `\n` itself before EOF.)
5. `FormatQuestion` **is exported** — pure function; tests pin byte order. Rationale: lets the anti-bluff (d) test (preview before label) check the function output WITHOUT going through `Prompt` — keeps the test focused. The function is also reusable from a future `/ask` slash command (F19.5) if added.
6. **Single** `preview` **string vs claude-code's** `QuestionPreview{type, content, title}` — v1 collapses the typed preview to a single string. Rationale: typed previews require a renderer that knows about diff / image / markdown rendering; F19's renderer (F18's `render.RenderLines`) is plain-text-only. Adding type dispatch would couple F19 to a markdown library / image protocol negotiation. Deferred to F19.5.
7. **Tool returns** `map[string]interface{}` **not** `Result` **struct** — matches existing tool return convention (F14 `shell_sandboxed` returns `map[string]interface{}`; F15 `task` returns `map[string]interface{}`; F17 `smart_edit` returns `map[string]interface{}`). Lets the LLM-side JSON decoder see a stable shape without the agent needing to know about `askuser.Result`. The internal `Result` struct is for type-safety inside the prompter.
8. **Non-TTY short-circuit is BEFORE any read** — proven by the byte-0-reader-position assertion in §5.2 (b) and Challenge Phase B/C. A bluffy alternative (read first, then validate) would block in non-TTY mode; the assertion catches it.
9. **Retry counter resets on each** `Prompt` **call** — three invalids on one prompt = error; the agent re-asking with a new prompt gets a fresh budget. Rationale: the budget is per-question, not per-session. A user who fat-fingered three times on one question deserves a fresh budget on the next question. Documented.
10. **Goroutine-leak on cancellation is acceptable v1** — `bufio.ReadString('\n')` blocks on the kernel; ctx-cancel returns from the prompter but the read goroutine stays parked until the next byte. Rationale: `os.Stdin` is process-lifetime; the leaked goroutine exits on process exit. v2 may use `os.Stdin.SetReadDeadline` on supported platforms (linux/macos via the `*os.File.SetReadDeadline` method). Documented in §4.6.
11. **Renderer is shared with F18** — `StdinPrompterOptions.Renderer` defaults to a fresh factory-constructed renderer, but the CLI can pass `c.renderer` to share. Rationale: a single renderer instance keeps the F18 viewport state coherent (so a tool-result-block render before the prompt doesn't get clobbered by the prompter's own RenderLines call). Documented as polish; correctness does not depend on it in v1.
12. **ANSI /** `\r` **handling is delegated to F18** — the prompter calls `render.RenderLines`; the renderer's plain-mode strips `\r` and the fancy-mode emits ANSI. F19 does NOT re-implement either. Rationale: F18 already passes the zero-CR-in-plain-mode invariant; F19 reuses it via composition.